

CAPSTONE PROJECT

AI-Powered Transaction Processing Pipeline

A four-agent AI workflow: Specification → Code Generation → Unit Tests → Documentation

Created by Artem Saienko

Project Overview

- A file-based pipeline that **validates**, **fraud-scores**, and **settles** transactions
- Every stage communicates only through JSON files under `shared/` — no in-process calls between stages
- Built entirely by four collaborating AI agents, each with a fixed scope and a durable rulebook (`agents.md`)
- Ships with a Flask dashboard, a custom MCP server, two Claude Code skills, and a coverage-gate push hook

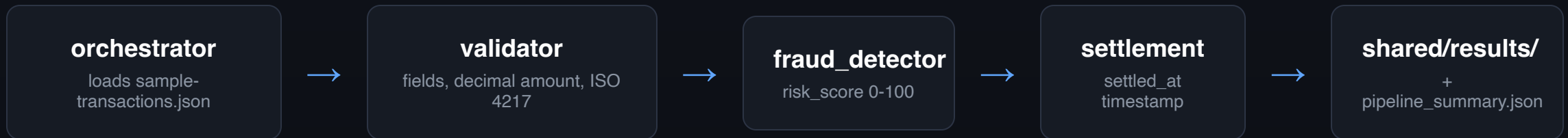
Why this matters

Financial pipelines need auditability and precision: exact decimal math, ISO 4217 currency checks, PII masking, and a full log trail per transaction — enforced the same way regardless of which agent touches the code.

The Four-Agent Workflow

Agent	Role	Produces	Plus
1 — Specification	Defines pipeline behavior	specification.md, agents.md	Skill: /write-spec
2 — Code Generation	Builds pipeline + front-end	orchestrator.py, pipeline/*.py, frontend/	MCP: context7 (2+ queries logged)
3 — Unit Tests	Verifies behavior + coverage	tests/* (29 tests)	Hook: blocks push under 80% coverage
4 — Documentation	Explains the finished system	README.md, HOWTORUN.md	Author line + architecture diagram

Pipeline Architecture



Intermediate hops (input/ → processing/ → output/) use a JSON envelope (message_id, source_stage, target_stage, data).
Terminal records in shared/results/ are flat, so the MCP server and front-end can read status directly.

Fraud Detection Logic

Scoring factors (flag threshold: 50)

- Amount over \$10,000 absolute value → **+60**
- Cross-border (metadata.country ≠ domestic market) → **+30**
- Off-hours timestamp (00:00–05:00 UTC) → **+30**

Weights are a documented, adjustable default — flagged as a [PLACEHOLDER] business rule in specification.md, not an invented hard requirement.

Verified against real sample data

- **TXN002 / TXN005** — high-value wires (\$25k / \$75k) → flagged
- **TXN004** — €500 transfer, 02:47 UTC, Germany → flagged (cross-border + off-hours)
- **TXN003** — \$9,999.99, just under threshold → correctly passes
- **TXN007** — negative amount, but a *refund* → correctly valid

Demo: Pipeline Run

Demo: Front-End Dashboard

homework-6 - python3 orchestrator.py

```
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN001",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN002",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN003",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN004",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN005",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN006",
  "outcome": "rejected:invalid_currency",
  "source_account": "...1006"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN007",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "validator",
  "transaction_id": "TXN008",
  "outcome": "validated"
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN001",
  "outcome": "passed",
  "risk_score": 0
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN002",
  "outcome": "flagged",
  "risk_score": 60,
  "risk_factors": ["high_value"]
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN003",
  "outcome": "passed",
  "risk_score": 0
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN004",
  "outcome": "flagged",
  "risk_score": 60,
  "risk_factors": ["cross_border", "off_hours"]
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN005",
  "outcome": "flagged",
  "risk_score": 60,
  "risk_factors": ["high_value"]
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN007",
  "outcome": "passed",
  "risk_score": 30
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "fraud_detector",
  "transaction_id": "TXN008",
  "outcome": "passed",
  "risk_score": 0
}
{
  "timestamp": "2026-07-08T11:14:16Z",
  "stage": "settlement",
  "transaction_id": "TXN001",
  "outcome": "settled"
}
```

Testing, Coverage & MCP Integration

```
homework-6 - pytest --cov=pipeline --cov=orchestrator

$ pytest --cov=pipeline --cov=orchestrator --cov-report=term-missing -q
..... [100%]
===== tests coverage =====
_____ coverage: platform darwin, python 3.12.13-final-0 _____
```

Name	Stmts	Miss	Cover	Missing
orchestrator.py	48	8	83%	78-86
pipeline/__init__.py	0	0	100%	
pipeline/common.py	40	0	100%	
pipeline/fraud_detector.py	53	0	100%	
pipeline/settlement.py	23	0	100%	
pipeline/validator.py	70	7	90%	119-127
TOTAL	234	15	94%	

```
29 passed in 0.10s

Required coverage gate: 80% → Achieved: 94% ✓
```

```
MCP servers - context7 + pipeline-status

## context7 query - library resolution + doc lookup
> mcp_context7_resolve-library-id(libraryName="Python decimal", query="rounding and quantizing Decimal for monetary values")
Library ID: /python/cpython (Source Reputation: High, Benchmark Score: 82.11)

> mcp_context7_query-docs(libraryId="/python/cpython", query="decimal module quantize ROUND_HALF_UP for currency amounts")
### Round Decimal to Two Places with quantize
Decimal('3.214').quantize(TWOPLACES)

### ROUND_HALF_UP
Round to nearest with ties going away from zero.

Applied in pipeline/validator.py + pipeline/fraud_detector.py:
Decimal(str(amount)) constructed directly from the JSON string (never float).

## custom MCP server - mcp/server.py (pipeline-status)
>>> get_transaction_status("TXN004")
{
  "transaction_id": "TXN004",
  "status": "flagged",
  "risk_score": 60,
  "risk_factors": ["cross_border", "off_hours"],
  "currency": "EUR",
  "amount": "500.00"
}

>>> list_pipeline_results()["counts_by_status"]
{ "settled": 4, "flagged": 3, "rejected": 1 }

>>> resource: pipeline://summary
{ "total": 8, "counts_by_status": { "settled": 4, "flagged": 3, "rejected": 1 } }
```

Lessons Learned

What worked well

- Splitting agents by strict file-ownership (spec/code/tests/docs) kept scope creep out — each agent had a one-line "never touch" list
- The integration test caught a real bug: terminal envelopes weren't deleted from shared/output/, leaving stale files behind
- Marking unresolved business rules (fraud weights) as [PLACEHOLDER] in the spec avoided silently inventing thresholds

Challenges & fixes

- Discovered a data-contract mismatch: the pre-built MCP server expected flat records in shared/results/, not envelopes — fixed pipeline stages to match
- High-value weight (+40) alone didn't clear the 50-point flag threshold — retuned to +60 and re-verified against the spec's stated outcomes
- Local mcp/ package name collides with the installed mcp SDK on import — not an issue since the server runs standalone as a subprocess